Individual Challenge num 2. Control Unit

Leonardo Martínez Peña - A01669629

***Abstract*— This work presents de design and functional validation of a multicycle processor control system implemented in VHDL. The project includes the development of a Main Control Unit modeled as a finite state machine and an ALU Control Unit responsible for decoding arithmetic and logical operations. The architecture follows the classical multicycle control approach and supports R-type, Load Word (LW), Store Word (SW), and Branch Equal (BEQ) instructions. Functional simulations were performed using Intel Quartus with manually defined testbench stimuli to verify state transitions, control signal generation, and ALU behavior. The integration of a previously designed Arithmetic Logic Unit enabled behavioral validation beyond signal level verification. Simulation results confirm correct execution sequencing and proper activation of control signals, demonstrating that the implemented design accurately replicates the behavior of a realistic multicycle processor control subsystem.**

I. Introduction

Finite State Machines (FSMs) play a fundamental role in the disign of digital systems and modern processor architectures. In multi-cycle processor implementations, the control unit is responsible for orchestrating the executrion of intructions by generation the appropriate control signals at each cycle. This sequential behavior makes FSM-base control design a natural and efficent solution.

Modern Reduced Instruction Set Computer (RISC) architecture, such as RISC-V, rely on clearly defined control logic to manage instruction fetch, decode, executrion, memory access, and write-back stages. Although modern commercial processors employ highly optimized and deeply pipelined control units, educational multicycle architectures offer a simplified yet accurate representation of real control behavior, while keeping implementation complexity manageable.

This project focuses on the design and implementation of two fundamental control components:

1. The Main Control Unit (CU), implemented as a multicycle finite state machine.
2. The ALU Control Unit, responsible for decoding the ALU operation based on control signals and function fields.

Both units were discribed using VHDL and validate through functional simulation in Intel Quartus. Additionally, a previously designed Arithmetic Logic Unit (ALU) was integrated into the system in order to verify the correct operation of the generated control signal, bringing the design closer to a realistic processor subsystem.

The objective of this work is to develop a hardware description model of a processor control system that closely resembles real world architectures.

II. Methods

The proposed control system was implemented using VHDL following a multisicycle architecture inspired by the methodology presented in Patterson and Hennessy. The design was divided into three main modules: the Main Control Unit (CU), the ALU Control Unit, and an Arithmetic Logic Unit (ALU) used for validation purposes. All modules were integrated in a top level entity to verify correct signal interaction.

A. Main Control Unit Design

The Main Control Unit was implemented as a finite state machine (FSM) with nine states (S0 – S8). The FSM control the sequential execution of instruction by generating appropriate control signals at each clock cycle.

The state transitions are determined by the system clock, reset signal, and instruction opcode. The following instruction types were considered:

- R-type instructions
- Load Word (LW)
- Store Word (SW)
- Branch Equal (BEQ)

The states were defined as follows:

- S0 (Instruction Fetch): Activates memory read and instruction register write signals while updating the program counter.
- S1 (Instruction Decode): Decodes the opcode and determines the next execution stage.
- S2 (Memory Address Computation): Computes effective address for memory instructions.
- S3 (Memory Read): Reads data from memory (LW).
- S4 (Write Back -LW): Writes memory data into the register file.
- S5 (Memory Write -SW): Writes register data into memory.
- S6 (Execution R-type): Performs ALU operation for arithmetic/logical instructions.
- S7 (Write Back R-type): Writes ALU result into the register file.
- S8 (Branch Completion): Performs branch comparison and conditionally updates the program counter.

At each state, default values were assigned to all control outputs to prevent unintended latch inference. Only the required control signals were asserted in each state. This approach ensures deterministic and stable FSM behavior.

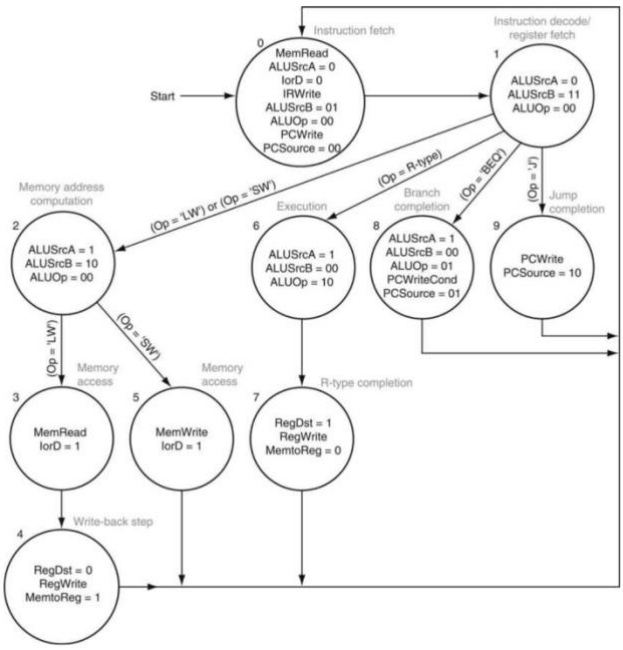


Fig. 1. Finite-state diagram for multicucle control (reproduced from [1], p. 1402).

The overall structure of the multicycle control unit follows the finite-state model ilustrated in Fig. 1, originally presented in [1].

As shown in Fig. 1, the original multicycle control model includes a jump completion state (State 9). In this implementation, State 9 was not required since jump instructions were not par of the defined instruction subset. All remaining states (S0 – S8) were implemented following the specified control signal behavior described in the reference model.

B. ALU Control Unit Design

The ALU Control Unit was implemented as a combinational module. It´s purpose is to transate the 2 bit ALUOp signal generated by the Main Control Unit, together with a 6 bit function field, into a 3 bit control signal the selects the specific ALU operation.

The ALUOp signal determines the operation category:

- 00: Addition (used for LW and SW address computation).
- 01: Subtraction (used for BEQ comparison).
- 10: Function based decoding (R-type instructions).

When ALUOp equals 10, the function field determines the specific arithmetic or logical operation, including addition, subtraction, AND, OR, NOR, and set less than. This hierarchical control scheme closely resembles real processor implementations, where the main controller generates high level control signals and a secondary decoder refines the ALU operation.

C. ALU Integration for Validation

A previously designed 4 bit Arithmetic Logic Unit was integrated to validate the behavior of the control system. The ALU supports arithmetic operations (addition and subtraction), logical operations (AND, OR, NOR), and comparison (set less than).

The ALU Control Unit generates a 3 bit selection signal that directly drives the ALU operation input. By observing the ALU output and carry signal during simulation, it was possible to verify that the control units correctly generated operation codes corresponding to the active instruction.

This integration enabled functional validation at a deeper level than basic signal inspection, providing behavioral confirmation of proper control sequencing.

D. Functional Simulation

Functional simulations were performed using Intel Quartus. Input stimuli for opcode, function field, and ALU operands were manually defined within the testbench environment. The clockand reset signals were also generated in simulation to abserve state transitions.

Each instruction type (R-type, LW, SW, and BEQ) was evaluated by applying the corresponding opcode and function field values. The simulation focused on verifying:

- Correct FSM state transitions
- Proper activation of control signals
- Acurate ALU operation selection
- Consistency of write back and memory control signals

The simulation environment allowed observation of all relevant internaland output signals to confirm proper system functionality.

III. RESULTS

This section presents the functional simulation results obtained using Intel Quartus. The objective of these simulations was to validate the correct operation of the multicycle control unit and the ALU control logic under different instruction types. Each instruction was evaluated independently with fixed input stimuli in order to observe state transitions, control signal activation, and ALU behavior. The analysis focuses on verifying that the generated control signals correspond to the expected stages of execution fetch, decode, execute, memory access, and write back, according to the multicycle architecture described in the methodology section.

R-type ADD validation:

During the R-type instruction test (opcode = 000000), the function field was set to function = 100000 to select the ADD operation. Operand inputs were fixed at A = 0011 (3) and B = 0101 (5). The control unit asserted ALUOp = 10 during the execute stage, enabling function base decoding in the ALU control Unit. As a result, the ALU output produced ALUres =

1000 (8), matching the expected sum. In the subsequent cycle, RegDst = 1 and RegWrite = 1 were asserted, confirming correct write back behavior for R-type instructions.

The complete waveform corresponding to this simulation is provided in Appendix A.

Load Word (LW) validation:

For the LW instruction (opcode = 100011), operands were fixed at A = 0001 (1) and B = 0011 (3). During the address computation stage, the control unit asserted ALUOp = 00, selecting the addition operation in the ALU. The resulting output was ALURes = 0100 (4), corresponding to the expected effective address (1 + 3 = 4). In the memory access stage, MemRead and IorD were activated, confirming correct data memory access behavior. Finally, during the write back stage, the control signals enabled register update through RegWrite and MemtoReg, validating proper execution of the LW instruction in the multicycle control architecture.

The complete waveform corresponding to this simulation is provided in Appendix B.

Store Word (SW) validation:

For the SW instruction (opcode = 101011), operands were set to A = 0010 (2) and B = 0100 (4). During the address computation stage, the control unit asserted ALUOp = 00, selecting the addition operation. The ALU produced ALURes = 0110 (6), corresponding to the expected affective address (2 + 4 = 6). In the memory access stage, MemWrite and IorD were asserted, enabling data storage to memory. As expected, RegWrite remained inactive, confirming that no register update occurs during the execution of the SW instruction. The observed control sequence validates correct multicycle operation for memory write instructions.

The complete waveform corresponding to this simulation is provided in Appendix C.

Branch Equal (BEQ) validation:

For the BEQ instruction (opcode = 000100), operands were set to A = 0100 (6) and B = 0100 (6). During the branch stage, the control unit asserted ALUOp = 01, selecting subtraction to compare register values. The ALU produced ALUres = 0000 (0), confirming equality (4 – 4 = 0). Simultaneously, PCWriteCond was activated and PCSource = 01 was selected, indicating conditional program counter update. The observed signal sequence validates correct branch control behavior in the multicycle control architecture.

The complete waveform corresponding to this simulation is provided in Appendix D.

IV. Conclusion

This work presented the design and functional validation of a multicycle processor control system impemented in VHDL. The Main Control Unit and ALU Control Unit were modeled as finite state machines following the methodology described by Patterson and Hennessy. Functional simulations performed in Intel Quartus confirmed correct state transitions and proper activation of control signals for R-type, LW, SW, and BEQ instructions.

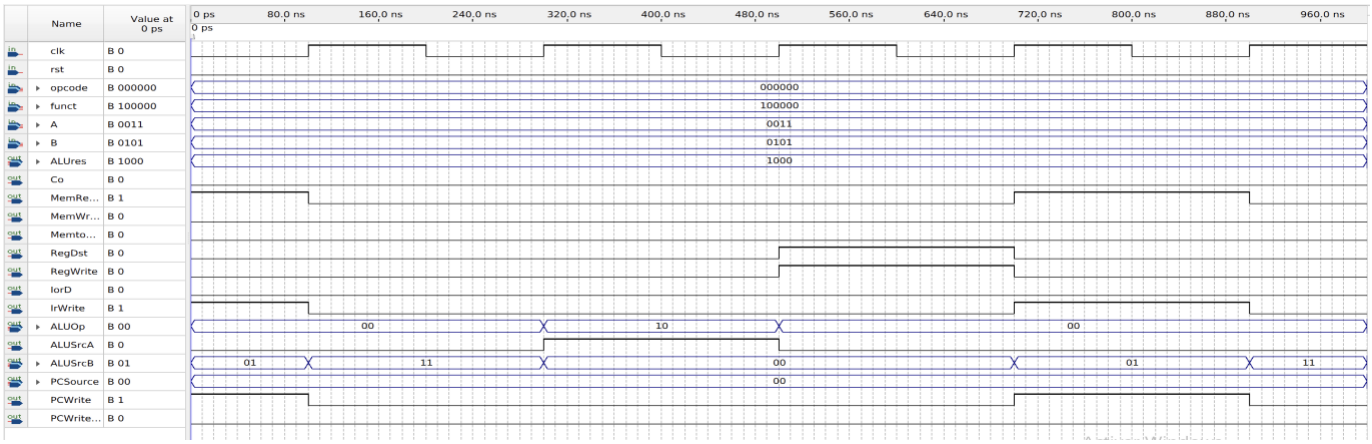
The integration of a previously designed Arithnetic Logic Unit allowed behavioral validation beyond signal level verification, demonstrating correct arithmetic and logical executrion during the appropiate stages. The results show that the implemented architecture closely replicates the behavior of a reaslistic mylticycle processor control subsystem.

Overall, this project represented a significant technical challenge that required careful analysis and iterative refinement. Through discussion and exchange of ideas with classmates, the design approach became progressively clearer, allowing conceptual obstacles to be resolved in a structured manner. Developing a multicycle control unit from specification to functional validation strengthened my undestanding of finite state machines, control sequencing, and hardware description methodologies. The project not only reinforced theoretical concepts from computer organization but also demonstrated the practical complexity involved in translating architectural models into working digital systems.

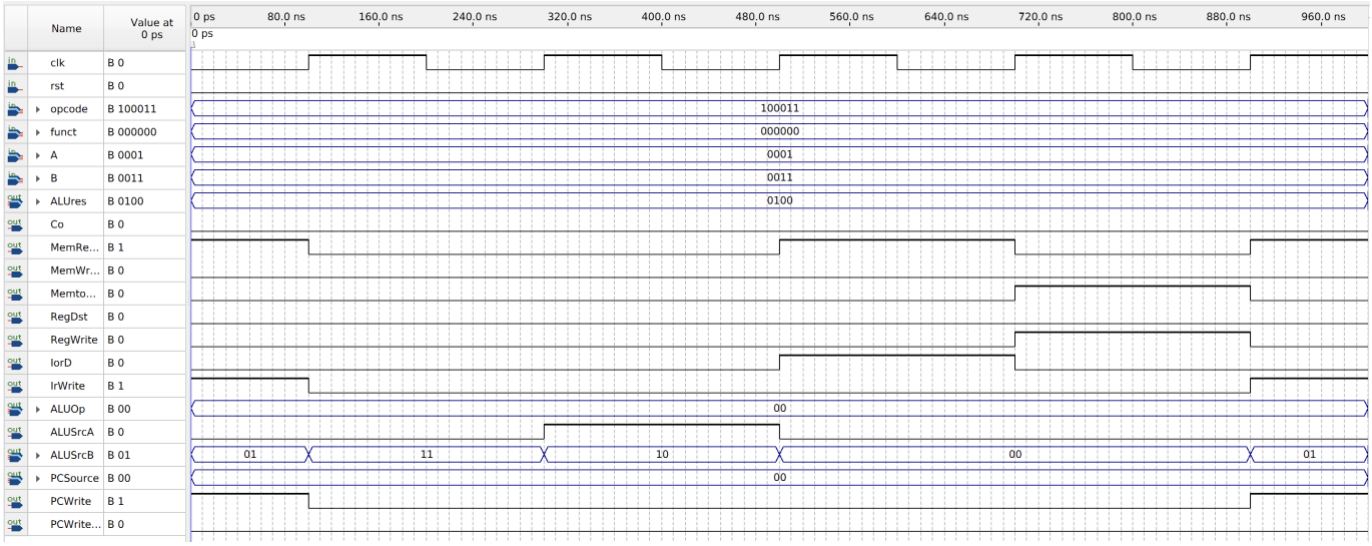
References

[1] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Burlington, MA, USA: Morgan Kaufmann, 2018.

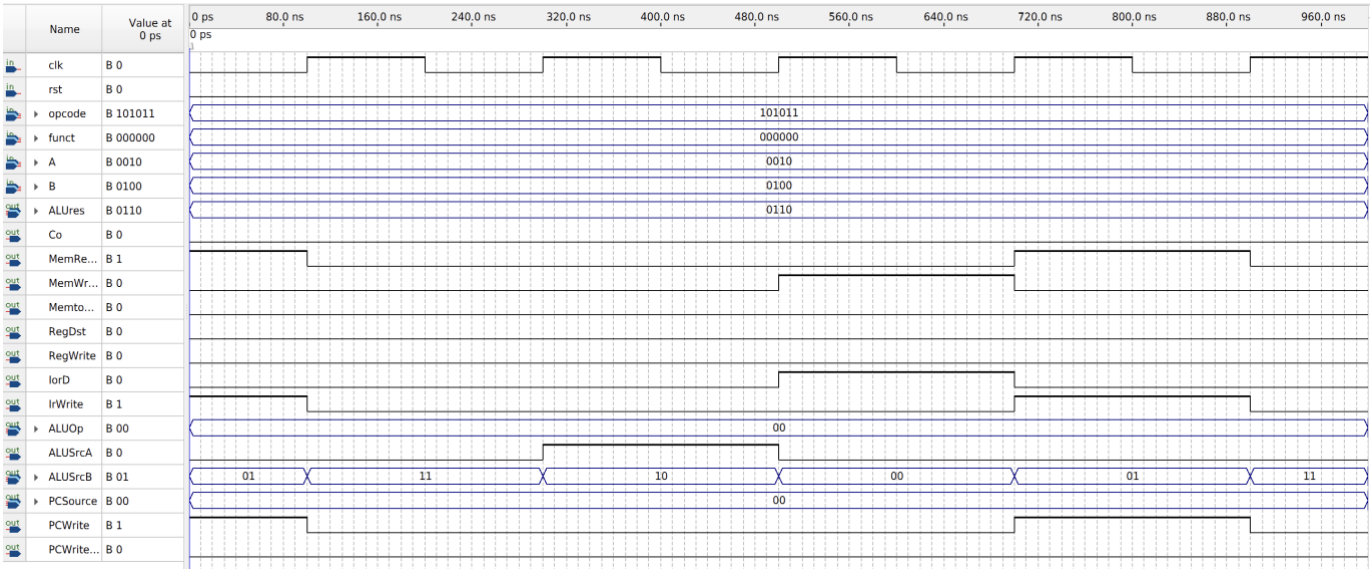
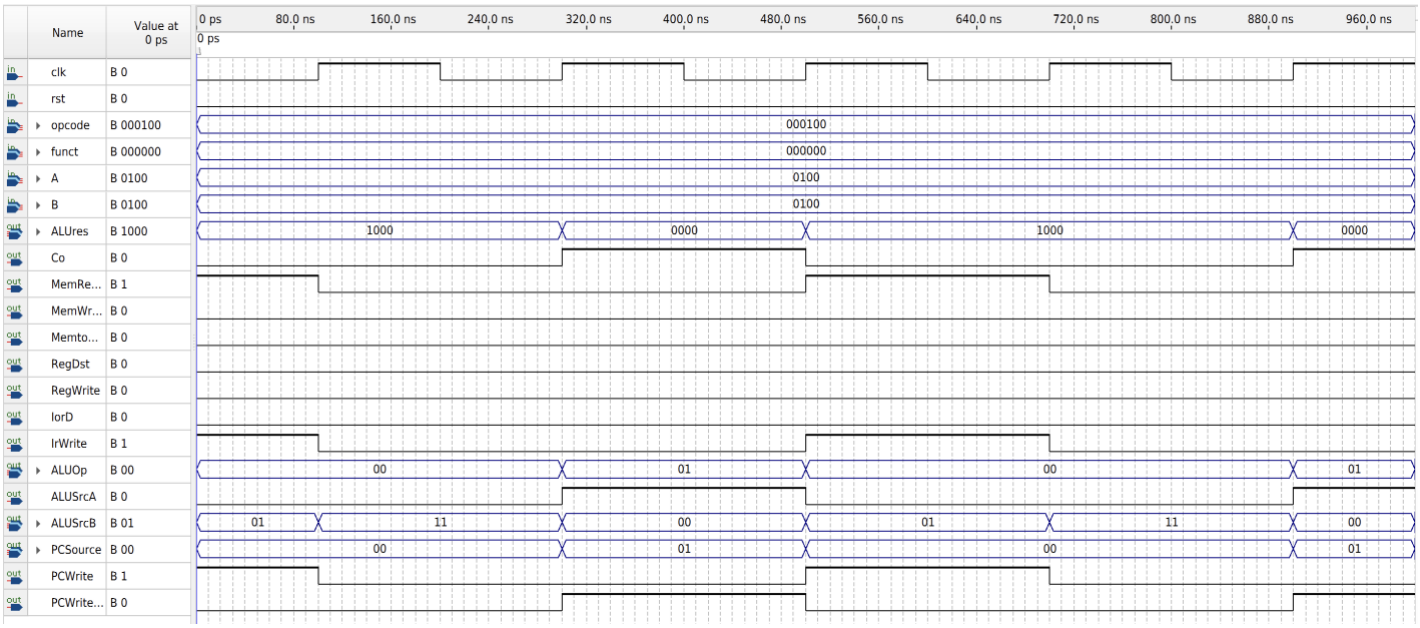
Appendix A. Wave simulation for R-type opcode



Appendix B. Wave simulation for Load Word (LW) opcode



Appendix C. Wave simulation for Save Word (SW) opcode.



Appendix D. Wave simulation for Branch Equal (BEQ) opcode.